

۱- بخش اول - ساخت AST

در این فاز قرار است با نوشتن semantic Action ها، Abstract Syntax Tree یا AST ساخته شود تا در مراحل بعد از آن استفاده کنید.
به نکات زیر در نوشتن کدهای خود دقت کنید:

- حتماً کلاس‌های مناسب برای هر قانون را با کمک ارث‌بری‌های مناسب تعریف کنید. تا حد امکان تفکیک گره‌ها را به کمک کلاس‌های متفاوت و ارث‌بری انجام دهید نه با نگه داشتن متغیرهایی مانند `enum type` و مشابه.
- پیاده‌سازی تابع `accept` را برای تمام کلاس‌ها فراموش نکنید. این کار برای `visit` کردن صحیح درخت ضروری است.

```
1 @Override
  public <T> T accept(IVisitor<T> visitor) {
3      return visitor.visit(this);
  }
```

- حتماً یک کلاس `abstract Node` داشته باشید که تمام کلاس‌ها از آن ارث‌بری می‌کنند.
- با کلیک روی نام کلاس و انتخاب، `vscode` به طور خودکار `setter` و `getter` ها را ایجاد خواهد کرد.
- استفاده از `debugger` داخلی `vscode` کمک زیادی به رفع اشکالات خواهد کرد.

۲- بخش دوم - Visitor

در این بخش قرار است برای تست AST یک ویزیتور ساده برای چاپ کردن برخی اطلاعات کد به کمک داده‌های AST بسازید. برای این کار یک کلاس `PrintVisitor` ایجاد کنید که از `abstract class Visitor<T>` ارث‌بری خواهد کرد. توجه داشته باشید تایپ خروجی `T` بسته به طراحی شما می‌تواند رشته، `void` یا یک کلاس `abstract` دیگر که طراحی می‌کنید باشد.
ویزیتور باید بعد از پایان ویزیت خود دقیقاً رشته‌ای با فرمت زیر را چاپ کند. هر سطح `indent` دقیقاً یک عدد کاراکتر `Tab = '\t'` می‌باشد.

```
program [modules:<num of modules in program> structs:<num of structs in program>]
2   module <Module_1 name> [methods:<num of methods in Module_1> fields:<num of
    fields in Module_1>]
      field <Field_1 Name> <Field_1 Type>:<private or public>
4     method <Method_1 Name> (<arg1 Type>*<arg2 Type>*...*<argN Type> -> <output
        Type>):<private or public> [statements:<num of stmts in Method_1>]
      struct <Struct_1 Name> [fields:<num of fields in Struct_1>]
6     field <Field_1 Name> <Field_1 Type>:<private or public>
```

به نکات زیر در نوشتن کدهای خود دقت کنید:

- ترتیب چاپ شدن ماژول/استراکت‌های برنامه اصلی به همان ترتیبی است که در کد آمده است.
- ترتیب چاپ شدن متد/فیلدها در هر ماژول یا استراکت به همان ترتیبی است که در کد آمده است.
- تایپ توابع به شکل زیر چاپ می‌شود:

```
(<arg1 Type>*<arg2 Type>*...*<argN Type> -> <output Type>)
```

برای بررسی خروجی مورد انتظار، یک فایل اجرایی Judge نیز در کنار پروژه در اختیار شما قرار داده می‌شود. در صورتی که بخواهید خروجی Judge برای یک فایل نمونه را مشاهده کنید، کافی است مسیر همان فایل نمونه را به این برنامه بدهید. برای مثال:

```
1 ./plc-judge [SAMPLE_PATH]
```

در صورتی که فایل اجرایی اجازه‌ی اجرا نداشت، ابتدا دستور زیر را اجرا کنید:

```
1 chmod +x plc-judge
```

سپس دوباره آن را اجرا کنید:
این فایل اجرایی برای محیط Linux x86_64 آماده شده است.

ورودی

```
1 module LoopTester begin
    private int stepCounter;
3
    public void reset() begin
5        this.stepCounter = 0;
    end
7
    public void runLoops() begin
9        this.reset();

11        mut int i;
        mut int j;
13        mut int sum = 0;

15        for (i = 0, j = 4; i < 3; i = i + 1, j = j - 1) begin
            output i;
17            output j;
            sum = sum + (i * j);
19        end

21        output sum;
        output this.stepCounter;
23    end
end
25
26 module Main includes LoopTester begin
27     public void main() begin
        this.runLoops();
29     end
end
31
32 struct Point begin
33     public float x;
        public float y;
35 end
36
37 module DataPacket begin
    private int id;
39     private int payload;

41     public void init(int i, int p) begin
        this.id = i;
43         this.payload = p;
    end
45
46     public int getPayload() begin
        return this.payload;
47     end
49
    public void setPayload(int p) begin
```

```
51         this.payload = p;
52     end
53 end
54
55 module OwnershipTest begin
56     public DataPacket processPacket(mut DataPacket packet) begin
57         mut int currentVal;
58         currentVal = packet.getPayload();
59     end
60 end
```

```

1 program [modules:4 structs:1]
2   module LoopTester [methods:2 fields:1]
3     field stepCounter int:private
4     method reset (void -> void):public [statements:1]
5     method runLoops (void -> void):public [statements:7]
6   module Main [methods:1 fields:0]
7     method main (void -> void):public [statements:1]
8   struct Point [fields:2]
9     field x float:public
10    field y float:public
11  module DataPacket [methods:3 fields:2]
12    field id int:private
13    field payload int:private
14    method init (int*int -> void):public [statements:2]
15    method getPayload (void -> int):public [statements:1]
16    method setPayload (int -> void):public [statements:1]
17  module OwnershipTest [methods:1 fields:0]
18    method processPacket (DataPacket -> DataPacket):public [statements:2]

```

۳- نکات تکمیلی

- در صورت هرگونه سوال با [عرفان](#) و [کسری](#) در ارتباط باشید.
- بخشی از نمره‌ی این پروژه بر اساس تست‌های خودکار محاسبه می‌شود. این تست‌ها به صورت خودکار روی کدهای تحویلی اجرا می‌شوند و نمره‌ی همان بخش بر اساس نتیجه‌ی اجرای آن‌ها تعیین خواهد شد.
- خروجی برنامه‌ی شما با خروجی Judge مقایسه می‌شود. در صورتی که خروجی‌ها متفاوت باشند، نمره‌ی آن تست به شما تعلق نمی‌گیرد و امکان ویرایش یا اصلاح کد پس از اجرای تست‌ها وجود نخواهد داشت.
- تمیزی کد، طراحی مناسب کلاس‌ها، و استفاده‌ی صحیح از inheritance در ساختار کلاس‌های AST بخشی از معیارهای نمره‌دهی این پروژه است.
- استفاده از Listener برای پیاده‌سازی این پروژه مجاز نیست و به منزله‌ی تقلب محسوب می‌شود.
- در صورت هرگونه تقلب، مطابق policy درس برخورد خواهد شد.
- هدف این تمرین یادگیری است؛ توصیه می‌شود پروژه را به صورت مستقل و توسط خودتان انجام دهید.